Spring Boot Guide

A practical reference document for the Spring AI RAG demonstration.

1. What is Spring Boot?

Spring Boot is a framework built on top of the Spring Framework that makes it easier to create production-ready Java applications. It provides sensible defaults, automatic configuration, starter dependencies, and an embedded web server.

A typical Spring Boot application can be started with a main class annotated with **@SpringBootApplication**. This annotation combines configuration, component scanning, and auto-configuration features.

2. Creating a Spring Boot Application

A new application can be created using Spring Initializr. Common project settings include Java, Maven, a group name, an artifact name, and dependencies such as Spring Web.

For a REST API application, the Spring Web dependency provides the components needed to create HTTP endpoints using annotations such as **@RestController**, **@GetMapping**, and **@PostMapping**.

3. Application Configuration

Spring Boot applications commonly use **application.properties** or **application.yml** for configuration. These files can define database settings, server settings, logging options, and application-specific properties.

For example, the default HTTP server port is **8080**. It can be changed by setting the server.port property.

```
server.port=8081
```

4. REST APIs

Spring Boot makes it straightforward to expose REST APIs. A class annotated with **@RestController** can contain methods that respond to HTTP requests.

For example, a GET endpoint can be mapped with **@GetMapping**. The method can return a String, Java object, collection, or another supported response type.

5. Spring Boot Profiles

Profiles allow different configuration values to be used for different environments. Common profiles include development, testing, and production.

A profile can be activated using the **spring.profiles.active** property. Profile-specific configuration can be stored in files such as **application-dev.yml** or **application-prod.yml**.

6. Dependency Injection

Dependency Injection is a core Spring concept. Instead of creating dependencies manually, Spring manages objects and provides the dependencies they require.

Constructor injection is generally a clear approach because required dependencies are provided when the object is created. Spring can automatically inject beans into constructors.

7. Application Port

By default, a Spring Boot web application runs on port **8080**. The port can be changed through application configuration.

For example, setting **server.port=9090** causes the embedded server to listen on port 9090 instead of port 8080.

8. Spring Boot Actuator

Spring Boot Actuator provides production-oriented features for monitoring and managing applications. It exposes endpoints that can provide information about application health, metrics, and other operational details.

The health endpoint is commonly used to determine whether an application is running correctly. Actuator endpoints should be configured carefully, especially in production.

9. Maven

Maven is a build and dependency management tool commonly used with Spring Boot. Dependencies are declared in the project's **pom.xml** file.

Maven can be used to compile the application, run tests, package the application, and manage external libraries.

10. Summary

Spring Boot simplifies Java application development by providing auto-configuration, starter dependencies, embedded servers, dependency injection, and production-ready features. It is widely used for building REST APIs, microservices, and enterprise applications.

RAG Demo Note: This document is intentionally focused on Spring Boot concepts. The RAG application can use this document as its knowledge source and answer questions based on the information contained here.

Example questions for testing: What is the default Spring Boot port? What annotation is used to create a REST controller? What is the purpose of Spring Boot profiles? What file is used to declare Maven dependencies?